路标识别

路标识别
- 效果介绍
- 例程源码
- 代码流程
- 代码结构
  1. 导入依赖
  2. 配置初始化
  3. 读取json参数
  4. 主函数
- 路标图片

效果介绍

本节我们将使用路标识别模型，来识别路标图片。

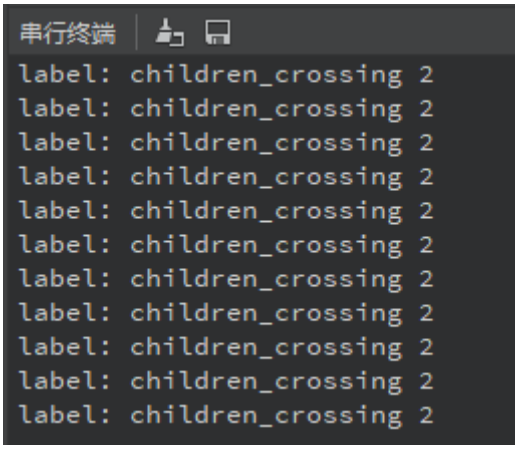
注意：由于模型训练时给出的路标图片是固定的，并且数据量有限，所以只能识别例程自带图片的路标，对于真实路标不一定能识别到。

使用CanMV IDE K230运行例程源码，可以看到摄像头画面，此时摄像头对着路标图片，可以看到框出路标并在方框上方显示路标名称。





同时canmv IDE的终端也会打印对应的名称。



例程源码

将以下源码复制到CanMV IDE K230上运行。

```
import os
import ujson
import aicube
from media.sensor import *
from media.display import *
from media.media import *
from time import *
import nncase_runtime as nn
import ulab.numpy as np
import time
import image
import gc
```

```
display_mode="lcd"
DISPLAY_WIDTH = ALIGN_UP(640, 16)
DISPLAY_HEIGHT = 480

OUT_RGB888P_WIDTH = ALIGN_UP(1080, 16)
OUT_RGB888P_HEIGH = 720

# 颜色盘
color_four = [(255, 220, 20, 60), (255, 119, 11, 32), (255, 0, 0, 142), (255, 0,
0, 230),
        (255, 106, 0, 228), (255, 0, 60, 100), (255, 0, 80, 100), (255, 0, 0,
70),
        (255, 0, 0, 192), (255, 250, 170, 30), (255, 100, 170, 30), (255, 220,
220, 0),
        (255, 175, 116, 175), (255, 250, 0, 30), (255, 165, 42, 42), (255, 255,
77, 255),
        (255, 0, 226, 252), (255, 182, 182, 255), (255, 0, 82, 0), (255, 120,
166, 157),
        (255, 110, 76, 0), (255, 174, 57, 255), (255, 199, 100, 0), (255, 72, 0,
118),
        (255, 255, 179, 240), (255, 0, 125, 92), (255, 209, 0, 151), (255, 188,
208, 182),
        (255, 0, 220, 176), (255, 255, 99, 164), (255, 92, 0, 73), (255, 133,
129, 255),
        (255, 78, 180, 255), (255, 0, 228, 0), (255, 174, 255, 243), (255, 45,
89, 255),
        (255, 134, 134, 103), (255, 145, 148, 174), (255, 255, 208, 186),
        (255, 197, 226, 255), (255, 171, 134, 1), (255, 109, 63, 54), (255, 207,
138, 255),
        (255, 151, 0, 95), (255, 9, 80, 61), (255, 84, 105, 51), (255, 74, 65,
105),
        (255, 166, 196, 102), (255, 208, 195, 210), (255, 255, 109, 65), (255,
0, 143, 149),
        (255, 179, 0, 194), (255, 209, 99, 106), (255, 5, 121, 0), (255, 227,
255, 205),
        (255, 147, 186, 208), (255, 153, 69, 1), (255, 3, 95, 161), (255, 163,
255, 0),
        (255, 119, 0, 170), (255, 0, 182, 199), (255, 0, 165, 120), (255, 183,
130, 88),
        (255, 95, 32, 0), (255, 130, 114, 135), (255, 110, 129, 133), (255, 166,
74, 118),
        (255, 219, 142, 185), (255, 79, 210, 114), (255, 178, 90, 62), (255, 65,
70, 15),
        (255, 127, 167, 115), (255, 59, 105, 106), (255, 142, 108, 45), (255,
196, 172, 0),
        (255, 95, 54, 80), (255, 128, 76, 255), (255, 201, 57, 1), (255, 246, 0,
122),
        (255, 191, 162, 208)]

root_path="/sdcard/mp_detect_guide/"
config_path=root_path+"deploy_config.json"
deploy_conf={}
debug_mode=0

class ScopedTiming:
    def __init__(self, info="", enable_profile=True):
```

```
        self.info = info
        self.enable_profile = enable_profile

    def __enter__(self):
        if self.enable_profile:
            self.start_time = time.time_ns()
        return self

    def __exit__(self, exc_type, exc_value, traceback):
        if self.enable_profile:
            elapsed_time = time.time_ns() - self.start_time
            print(f"{self.info} took {elapsed_time / 1000000:.2f} ms")

def read_deploy_config(config_path):
    # 打开JSON文件以进行读取deploy_config
    with open(config_path, 'r') as json_file:
        try:
            # 从文件中加载JSON数据
            config = ujson.load(json_file)

            # 打印数据（可根据需要执行其他操作）
            #print(config)
        except ValueError as e:
            print("JSON 解析错误:", e)
    return config

def detection():
    print("det_infer start")
    # 使用json读取内容初始化部署变量
    deploy_conf=read_deploy_config(config_path)
    kmodel_name=deploy_conf["kmodel_path"]
    labels=deploy_conf["categories"]
    confidence_threshold= deploy_conf["confidence_threshold"]
    nms_threshold = deploy_conf["nms_threshold"]
    img_size=deploy_conf["img_size"]
    num_classes=deploy_conf["num_classes"]
    nms_option = deploy_conf["nms_option"]
    model_type = deploy_conf["model_type"]
    if model_type == "AnchorBaseDet":
        anchors = deploy_conf["anchors"][0] + deploy_conf["anchors"][1] +
deploy_conf["anchors"][2]
    kmodel_frame_size = img_size
    frame_size = [OUT_RGB888P_WIDTH,OUT_RGB888P_HEIGH]
    strides = [8,16,32]

    # 计算padding值
    ori_w = OUT_RGB888P_WIDTH
    ori_h = OUT_RGB888P_HEIGH
    width = kmodel_frame_size[0]
    height = kmodel_frame_size[1]
    ratiow = float(width) / ori_w
    ratioh = float(height) / ori_h
    if ratiow < ratioh:
        ratio = ratiow
    else:
        ratio = ratioh
    new_w = int(ratio * ori_w)
    new_h = int(ratio * ori_h)
```

```
    dw = float(width - new_w) / 2
    dh = float(height - new_h) / 2
    top = int(round(dh - 0.1))
    bottom = int(round(dh + 0.1))
    left = int(round(dw - 0.1))
    right = int(round(dw - 0.1))

    # init kpu and load kmodel
    kpu = nn.kpu()
    ai2d = nn.ai2d()
    kpu.load_kmodel(root_path+kmodel_name)
    ai2d.set_dtype(nn.ai2d_format.NCHW_FMT,
                                    nn.ai2d_format.NCHW_FMT,
                                    np.uint8, np.uint8)
    ai2d.set_pad_param(True, [0,0,0,0,top,bottom,left,right], 0, [114,114,114])
    ai2d.set_resize_param(True, nn.interp_method.tf_bilinear,
nn.interp_mode.half_pixel )
    ai2d_builder = ai2d.build([1,3,OUT_RGB888P_HEIGH,OUT_RGB888P_WIDTH],
[1,3,height,width])

    # 初始化并配置sensor
    # sensor = Sensor(width=1280, height=960)
    sensor = Sensor()
    sensor.reset()
    # 设置镜像
    sensor.set_hmirror(False)
    # 设置翻转
    sensor.set_vflip(False)
    # 通道0直接给到显示VO，格式为YUV420
    sensor.set_framesize(width = DISPLAY_WIDTH, height = DISPLAY_HEIGHT)
    sensor.set_pixformat(PIXEL_FORMAT_YUV_SEMIPLANAR_420)
    # 通道2给到AI做算法处理，格式为RGB888
    sensor.set_framesize(width = OUT_RGB888P_WIDTH , height = OUT_RGB888P_HEIGH,
chn=CAM_CHN_ID_2)
    sensor.set_pixformat(PIXEL_FORMAT_RGB_888_PLANAR, chn=CAM_CHN_ID_2)
    # 绑定通道0的输出到vo
    sensor_bind_info = sensor.bind_info(x = 0, y = 0, chn = CAM_CHN_ID_0)
    Display.bind_layer(**sensor_bind_info, layer = Display.LAYER_VIDEO1)
    # 设置为ST7701显示，默认640x480
    Display.init(Display.ST7701, to_ide = True)


    #创建OSD图像
    osd_img = image.Image(DISPLAY_WIDTH, DISPLAY_HEIGHT, image.ARGB8888)
    try:
        # media初始化
        MediaManager.init()
        # 启动sensor
        sensor.run()
        rgb888p_img = None
        ai2d_input_tensor = None
        data = np.ones((1,3,width,height),dtype=np.uint8)
        ai2d_output_tensor = nn.from_numpy(data)
        while  True:
            with ScopedTiming("total",debug_mode > 0):
                rgb888p_img = sensor.snapshot(chn=CAM_CHN_ID_2)
                # for rgb888planar
                if rgb888p_img.format() == image.RGBP888:
```

```
                    ai2d_input = rgb888p_img.to_numpy_ref()
                    ai2d_input_tensor = nn.from_numpy(ai2d_input)
                    ai2d_builder.run(ai2d_input_tensor, ai2d_output_tensor)

                    # set input
                    kpu.set_input_tensor(0, ai2d_output_tensor)
                    # run kmodel
                    kpu.run()
                    # get output
                    results = []
                    for i in range(kpu.outputs_size()):
                        out_data = kpu.get_output_tensor(i)
                        result = out_data.to_numpy()
                        result =
result.reshape((result.shape[0]*result.shape[1]*result.shape[2]*result.shape[3])
)
                        del out_data
                        results.append(result)
                    gc.collect()

                    # postprocess
                    if model_type == "AnchorBaseDet":
                        det_boxes = aicube.anchorbasedet_post_process(
results[0], results[1], results[2], kmodel_frame_size, frame_size, strides,
num_classes, confidence_threshold, nms_threshold, anchors, nms_option)
                    elif model_type == "GFLDet":
                        det_boxes = aicube.gfldet_post_process( results[0],
results[1], results[2], kmodel_frame_size, frame_size, strides, num_classes,
confidence_threshold, nms_threshold, nms_option)
                    else:
                        det_boxes = aicube.anchorfreedet_post_process(
results[0], results[1], results[2], kmodel_frame_size, frame_size, strides,
num_classes, confidence_threshold, nms_threshold, nms_option)
                    osd_img.clear()
                    if det_boxes:
                        for det_boxe in det_boxes:
                            x1, y1, x2, y2 =
det_boxe[2],det_boxe[3],det_boxe[4],det_boxe[5]
                            w = float(x2 - x1) * DISPLAY_WIDTH //
OUT_RGB888P_WIDTH
                            h = float(y2 - y1) * DISPLAY_HEIGHT //
OUT_RGB888P_HEIGH
                            osd_img.draw_rectangle(int(x1 * DISPLAY_WIDTH //
OUT_RGB888P_WIDTH) , int(y1 * DISPLAY_HEIGHT // OUT_RGB888P_HEIGH) , int(w) ,
int(h) , color=color_four[det_boxe[0]][1:])
                            label = labels[det_boxe[0]]
                            print("label:", label, det_boxe[0])

                            score = str(round(det_boxe[1],2))
                            osd_img.draw_string_advanced( int(x1 * DISPLAY_WIDTH
// OUT_RGB888P_WIDTH) , int(y1 * DISPLAY_HEIGHT // OUT_RGB888P_HEIGH)-50,32,
label + " " + score , color=color_four[det_boxe[0]][1:])
                    Display.show_image(osd_img, 0, 0, Display.LAYER_OSD3)
                    gc.collect()
                rgb888p_img = None
    except Exception as e:
        print(f"An error occurred during buffer used: {e}")
    finally:
```

```
            os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)
            del ai2d_input_tensor
            del ai2d_output_tensor
            #停止摄像头输出
            sensor.stop()
            #去初始化显示设备
            Display.deinit()
            #释放媒体缓冲区
            MediaManager.deinit()
            gc.collect()
            time.sleep(1)
            nn.shrink_memory_pool()
        print("det_infer end")
        return 0


if __name__=="__main__":
    detection()
```

代码流程

主要工作原理：

1. 从摄像头获取实时视频流
2. 对每一帧进行预处理，调整尺寸以适配模型输入要求
3. 将处理后的图像送入神经网络进行推理
4. 根据置信度阈值筛选分类结果
5. 在显示界面上绘制识别结果
6. 循环处理下一帧

代码结构

1. 导入依赖

```
import os
import ujson
import aicube
from media.sensor import *
from media.display import *
from media.media import *
from time import *
import nncase_runtime as nn
import ulab.numpy as np
import time
import image
import gc
```

2. 配置初始化

```
root_path="/sdcard/mp_detect_guide/"
config_path=root_path+"deploy_config.json"
deploy_conf={}
debug_mode=0
```

其中，root_path指定了json参数和模型文件的路径。

3. 读取json参数

```
def read_deploy_config(config_path):
    # 打开JSON文件以进行读取deploy_config
    with open(config_path, 'r') as json_file:
        try:
            # 从文件中加载JSON数据
            config = ujson.load(json_file)

            # 打印数据（可根据需要执行其他操作）
            #print(config)
        except ValueError as e:
            print("JSON 解析错误:", e)
    return config
```

4. 主函数

初始化摄像头和显示屏等外设，然后读取摄像头画面，再传入kpu进行计算，与模型文件匹配，输出识别结果，再将识别结果绘制到显示屏上显示。

```

def detection():
    print("det_infer start")
    # 使用json读取内容初始化部署变量
    deploy_conf=read_deploy_config(config_path)
    kmodel_name=deploy_conf["kmodel_path"]
    labels=deploy_conf["categories"]
    confidence_threshold= deploy_conf["confidence_threshold"]
    nms_threshold = deploy_conf["nms_threshold"]
    img_size=deploy_conf["img_size"]
    num_classes=deploy_conf["num_classes"]
    nms_option = deploy_conf["nms_option"]
    model_type = deploy_conf["model_type"]
    if model_type == "AnchorBaseDet":
        anchors = deploy_conf["anchors"][0] + deploy_conf["anchors"][1] +
deploy_conf["anchors"][2]
    kmodel_frame_size = img_size
    frame_size = [OUT_RGB888P_WIDTH,OUT_RGB888P_HEIGH]
    strides = [8,16,32]

    # 计算padding值
    ori_w = OUT_RGB888P_WIDTH
    ori_h = OUT_RGB888P_HEIGH
    width = kmodel_frame_size[0]
    height = kmodel_frame_size[1]
    ratiow = float(width) / ori_w
```

```
    ratioh = float(height) / ori_h
    if ratiow < ratioh:
        ratio = ratiow
    else:
        ratio = ratioh
    new_w = int(ratio * ori_w)
    new_h = int(ratio * ori_h)
    dw = float(width - new_w) / 2
    dh = float(height - new_h) / 2
    top = int(round(dh - 0.1))
    bottom = int(round(dh + 0.1))
    left = int(round(dw - 0.1))
    right = int(round(dw - 0.1))

    # init kpu and load kmodel
    kpu = nn.kpu()
    ai2d = nn.ai2d()
    kpu.load_kmodel(root_path+kmodel_name)
    ai2d.set_dtype(nn.ai2d_format.NCHW_FMT,
                                    nn.ai2d_format.NCHW_FMT,
                                    np.uint8, np.uint8)
    ai2d.set_pad_param(True, [0,0,0,0,top,bottom,left,right], 0, [114,114,114])
    ai2d.set_resize_param(True, nn.interp_method.tf_bilinear,
nn.interp_mode.half_pixel )
    ai2d_builder = ai2d.build([1,3,OUT_RGB888P_HEIGH,OUT_RGB888P_WIDTH],
[1,3,height,width])

    # 初始化并配置sensor
    # sensor = Sensor(width=1280, height=960)
    sensor = Sensor()
    sensor.reset()
    # 设置镜像
    sensor.set_hmirror(False)
    # 设置翻转
    sensor.set_vflip(False)
    # 通道0直接给到显示VO，格式为YUV420
    sensor.set_framesize(width = DISPLAY_WIDTH, height = DISPLAY_HEIGHT)
    sensor.set_pixformat(PIXEL_FORMAT_YUV_SEMIPLANAR_420)
    # 通道2给到AI做算法处理，格式为RGB888
    sensor.set_framesize(width = OUT_RGB888P_WIDTH , height = OUT_RGB888P_HEIGH,
chn=CAM_CHN_ID_2)
    sensor.set_pixformat(PIXEL_FORMAT_RGB_888_PLANAR, chn=CAM_CHN_ID_2)
    # 绑定通道0的输出到vo
    sensor_bind_info = sensor.bind_info(x = 0, y = 0, chn = CAM_CHN_ID_0)
    Display.bind_layer(**sensor_bind_info, layer = Display.LAYER_VIDEO1)
    # 设置为ST7701显示，默认640x480
    Display.init(Display.ST7701, to_ide = True)


    #创建OSD图像
    osd_img = image.Image(DISPLAY_WIDTH, DISPLAY_HEIGHT, image.ARGB8888)
    try:
        # media初始化
        MediaManager.init()
        # 启动sensor
        sensor.run()
        rgb888p_img = None
        ai2d_input_tensor = None
```

```
        data = np.ones((1,3,width,height),dtype=np.uint8)
        ai2d_output_tensor = nn.from_numpy(data)
        while  True:
            with ScopedTiming("total",debug_mode > 0):
                rgb888p_img = sensor.snapshot(chn=CAM_CHN_ID_2)
                # for rgb888planar
                if rgb888p_img.format() == image.RGBP888:
                    ai2d_input = rgb888p_img.to_numpy_ref()
                    ai2d_input_tensor = nn.from_numpy(ai2d_input)
                    ai2d_builder.run(ai2d_input_tensor, ai2d_output_tensor)

                    # set input
                    kpu.set_input_tensor(0, ai2d_output_tensor)
                    # run kmodel
                    kpu.run()
                    # get output
                    results = []
                    for i in range(kpu.outputs_size()):
                        out_data = kpu.get_output_tensor(i)
                        result = out_data.to_numpy()
                        result =
result.reshape((result.shape[0]*result.shape[1]*result.shape[2]*result.shape[3])
)
                        del out_data
                        results.append(result)
                    gc.collect()

                    # postprocess
                    if model_type == "AnchorBaseDet":
                        det_boxes = aicube.anchorbasedet_post_process(
results[0], results[1], results[2], kmodel_frame_size, frame_size, strides,
num_classes, confidence_threshold, nms_threshold, anchors, nms_option)
                    elif model_type == "GFLDet":
                        det_boxes = aicube.gfldet_post_process( results[0],
results[1], results[2], kmodel_frame_size, frame_size, strides, num_classes,
confidence_threshold, nms_threshold, nms_option)
                    else:
                        det_boxes = aicube.anchorfreedet_post_process(
results[0], results[1], results[2], kmodel_frame_size, frame_size, strides,
num_classes, confidence_threshold, nms_threshold, nms_option)
                    osd_img.clear()
                    if det_boxes:
                        for det_boxe in det_boxes:
                            x1, y1, x2, y2 =
det_boxe[2],det_boxe[3],det_boxe[4],det_boxe[5]
                            w = float(x2 - x1) * DISPLAY_WIDTH //
OUT_RGB888P_WIDTH
                            h = float(y2 - y1) * DISPLAY_HEIGHT //
OUT_RGB888P_HEIGH
                            osd_img.draw_rectangle(int(x1 * DISPLAY_WIDTH //
OUT_RGB888P_WIDTH) , int(y1 * DISPLAY_HEIGHT // OUT_RGB888P_HEIGH) , int(w) ,
int(h) , color=color_four[det_boxe[0]][1:])
                            label = labels[det_boxe[0]]
                            print("label:", label, det_boxe[0])

                            score = str(round(det_boxe[1],2))
```

```
                            osd_img.draw_string_advanced( int(x1 * DISPLAY_WIDTH
// OUT_RGB888P_WIDTH) , int(y1 * DISPLAY_HEIGHT // OUT_RGB888P_HEIGH)-50,32,
label + " " + score , color=color_four[det_boxe[0]][1:])
                    Display.show_image(osd_img, 0, 0, Display.LAYER_OSD3)
                    gc.collect()
                rgb888p_img = None
    except Exception as e:
        print(f"An error occurred during buffer used: {e}")
    finally:
        os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)
        del ai2d_input_tensor
        del ai2d_output_tensor
        #停止摄像头输出
        sensor.stop()
        #去初始化显示设备
        Display.deinit()
        #释放媒体缓冲区
        MediaManager.deinit()
        gc.collect()
        time.sleep(1)
        nn.shrink_memory_pool()
    print("det_infer end")
    return 0
```

路标图片

go_straight：



turn_right：



honking：



pedestrian_crossing：



speed_limit：



stop:



children_crossing:



parking-A:



parking-B:

